To generate all the indices for the bits used to store a key, we go through a two-step process.

Keep in mind that our ultimate goal is to transform a string into *k* positions, between 0 and `m - 1`.

First, we use two hash functions on strings very different from each other: *Murmur hashing* and *fnv1 hashing*. The chances that, for a given string, both of them produces the same result are beyond slim.

Then, for each of the `k` bits we have to store, we retrieve the corresponding hash function in our pool. For each position `I` between `0` and `k-1` we have generated (on initialization) a *double hashing* function h_i: the `i`-th bit will therefore be returned by $h_i(h_M, h_F)$, where h_M is the result of murmur hashing on the input key, and h_F the result of fnv1 hashing.

Although the highest level of randomization would be obtained with a random seed for each run, we need to have a way to force a deterministic behavior both for tests, and to be able to recreate a Bloom filter that can make sense of a given buffer, in case the filter is serialized, or to support quick restart over failure: therefore, we should also leave the option to pass the seed to the Bloom filter's constructor.

Listing 4.7 Method key2Positions

```
function key2Positions(hashFunctions, seed,
key)                                  #1
  hM ← murmurHash32(key, seed)
                                      #2
```